Together First, Apart Later? When Optimizers Start to Learn Different Functions

Initial checkpoint, MIT 6.7910, Fall 2026

Juntang Wang and Víctor Conchello · October 30, 2026

Question. From the same initialization and the same sequence of mini-batches, do SGD with momentum, AdamW and Muon first follow the same path in function space and only later separate? If so, where is the split, what coincides with it (an edge-of-stability event (Cohen et al., 2021; Cohen et al., 2022), the end of warmup, a loss level), and is the final function already decided there?

What we have done. We trained nanoGPT (Karpathy, 2022) on character-level Shakespeare (10.8M parameters) and TinyStories-10M and -30M (14M and 30M parameters) with SGD, AdamW and Muon (Jordan et al., 2024), at learning rates from a sweep, from a shared initialization and with five data orders per optimizer. Along training we logged the next-token probability distributions on fixed held-out contexts, and the top eigenvalue of the Hessian and, for AdamW, of the preconditioned Hessian (Cohen et al., 2022), both computed with the Lanczos method (1950). We swapped optimizers at nine fork steps with warm optimizer states, measured linear mode connectivity, and replicated the CIFAR-10 break-even analysis (Jastrzebski et al., 2020). In total 392 runs, about 103 GPU-hours.

Method. The distance between two runs is the mean Jensen–Shannon divergence of their next-token predictions. We compare runs at equal *training loss*, so that one optimizer being faster is not counted as a difference, and divide by a noise floor: the distance between two runs of the same optimizer that differ only in data order. A split is a stretch of loss levels where this ratio exceeds 1.5; it is persistent if it lasts to the end of training, with a bootstrap over data orders for its location.

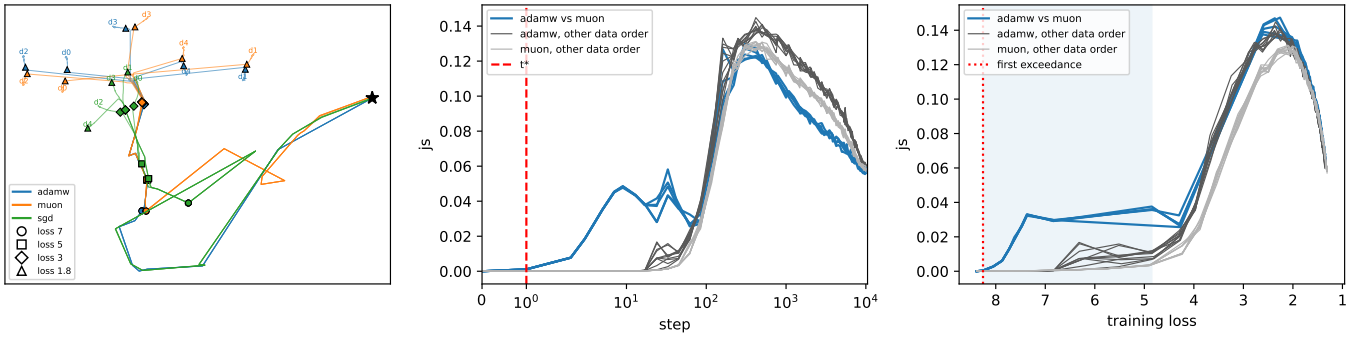


Figure 1: TinyStories-10M. Left: the training trajectories of all 15 runs (three optimizers, data orders d0–d4 labelled at the end of each run) in prediction space, as in Fig. 1 of Jastrzebski et al. (2020): UMAP of the Jensen–Shannon distance (the square root of the divergence) between the predictions of every pair of logged steps. Star: the shared initialization; markers: the first step at which each run reaches the given training loss. UMAP keeps neighbourhoods but not distances. Middle: distance between AdamW and Muon at equal steps (blue) and the data-order floor (grey). Right: the same pairs at equal training loss; shaded, the episode above the floor. Hyperparameters in Appendix A.1.

Result 1: apart first, together later (Figs. 1 and 2). At equal steps the optimizers differ from the first steps, mostly because they progress at different speeds. At equal loss, in all three settings and for every pair, the distance is 30–200 times the floor at high loss and falls back to about the floor around the stage at which the models have learned the bigram statistics of the data. On TinyStories it stays there: at the end of TinyStories-30M, replacing AdamW by Muon changes the predictions no more than changing the data order. The trajectories (Fig. 1, left) show the same: the paths of the three optimizers separate early, run together from about loss 5, and at the end fan out by data order, with AdamW and Muon of the same data order ending side by side. Of the nine pairs only Muon–SGD on Shakespeare, where SGD barely trains, has a persistent split, and it is lost with another metric or a stricter threshold. The answer to the title is therefore the reverse of the hypothesis: the optimizers' functions differ early and meet later.

Result 2: no edge-of-stability event at the split. On TinyStories AdamW's preconditioned sharpness sits at its threshold $(2+2\beta_1)/((1-\beta_1)\eta)$ for much of training, whether or not the optimizers differ, and SGD reaches $(2+2\mu)/\eta$ only early in training. The early episodes end within warmup (steps 9–85), the same way for AdamW–Muon, where SGD is not involved. A stage of learning describes the re-convergence better than a stability event.

Result 3: the last optimizer decides the function (Fig. 3). When we switch optimizer at step t, the fork ends within the floor of the new optimizer whenever that optimizer is fast enough to get there in the remaining steps: forks from SGD to AdamW or Muon land on the new optimizer even after 4000 of 10,000 steps, while forks into the slower

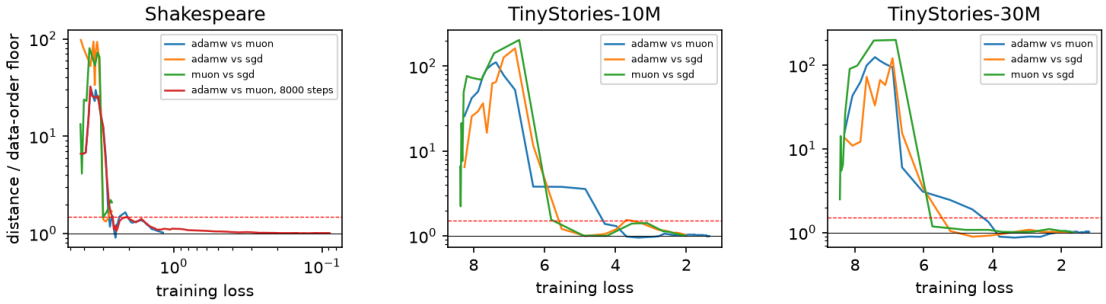


Figure 2: Loss-matched distance between optimizers over the data-order floor (training progresses to the right); red dashed, the threshold 1.5. Red, left: AdamW–Muon trained into memorization (8000 steps). Hyperparameters in Appendix A.2.

SGD end closer to their parent from step 1500 on (step 5000 on TinyStories-30M). The endpoint is thus not decided early. Weight space keeps a mark that function space does not: after a change of optimizer, a fork needs a later fork step to stay linearly connected to its parent (Frankle et al., 2020) than after a change of data order.

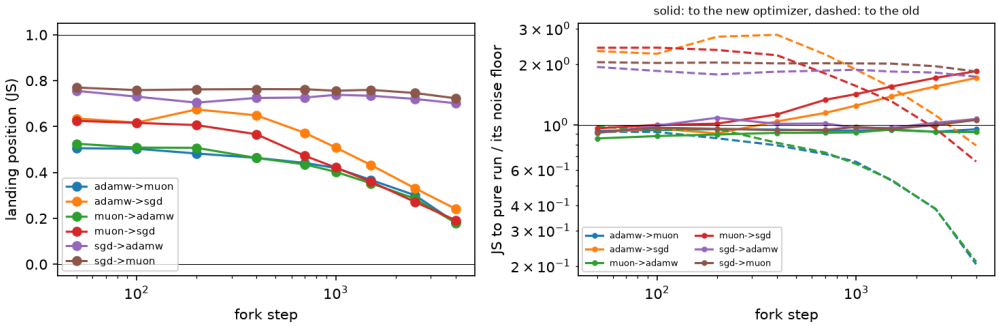


Figure 3: Swaps on TinyStories-10M. Left: where the fork ends between pure A (0) and pure B (1). Right: its distance to pure B (solid) and to pure A (dashed) in units of their floors; below 1 means indistinguishable from that optimizer. Hyperparameters in Appendix A.3.

Result 4 (moonshot): the sharp direction at the edge of stability. Restricted to the steps at each optimizer's threshold and averaged over the five data orders, the top eigenvector of the operator that sets the threshold lies at the input end: the embeddings and first three blocks carry 75–90% of it in all settings. SGD's lies mostly in the first block; AdamW's, of $P^{-1/2}HP^{-1/2}$, in the embeddings on TinyStories (about 60%), where rare tokens make P^{-1} largest; Muon's, without a threshold, in its first three blocks (on Shakespeare in the embeddings, left to its auxiliary AdamW). Except in the embeddings, 66–83% of the eigenvalue comes from curvature *between* blocks. The direction changes block long after the optimizers have re-met, so it does not mark a split.

Result 5: a model of the results (proof sketch). Let $\mathcal{F}_s$ be the predictors that see only the last s tokens, with best element q_s. As $\sqrt{\mathrm{JS}}$ is a metric, JS (bits) $\leq$ total variation, and by Pinsker, two predictors in $\mathcal{F}_s$ with excess losses $\Delta L_A, \Delta L_B$ over q_s have $\mathbb{E}\,\mathrm{JS} \leq \sqrt{2}(\sqrt{\Delta L_A} + \sqrt{\Delta L_B})$, whatever the optimizer. So at equal loss the optimizers must meet near the bigram optimum; they can differ before it, where their update geometries take different routes (SGD passes through the unigram model, Muon skips it), or in a class without a best element, as in the linear control below. The break-even point depends on how the update scales with the gradient: $(2+2\mu)/\eta$ for SGD; for AdamW the threshold applies to $P^{-1}H$, so the raw sharpness there scales as the gradient over η, as we find (log-log slope 1.15–1.6 on TinyStories); Muon's update is invariant to the gradient's scale and on a quadratic never diverges, so it has no break-even point on its matrices. Stability caps curvature but does not choose between functions of equal loss, hence Result 2.

Pipeline check and controls. On CIFAR-10 the break-even effects replicate: the sharpness levels off at or below $2/\eta$ and lower for larger rates, and the gradient covariance is better conditioned; but at equal loss two learning rates differ by at most about twice the floor. Controls: a second initialization reproduces the results, and on a linear model on separable data, where gradient descent and Adam have different max-margin limits (Soudry et al., 2018; Zhang et al., 2024), the same test does find a persistent split.

Next (Nov–Dec). (i) Moonshot: whether the sharp direction moves to the new optimizer's layers after a swap, and lowering the learning rate of only those layers. (ii) Sweeps with sharpness (running): Muon's should not scale as $1/\eta$. (iii) Robustness of the split rule to the metric and threshold. (iv) Presentation (Dec 1–4), paper, code and Colab notebook (Dec 5–11). Code: `https://github.com/qqgjyx/MIT-6.7910`.

References

Cohen, J. M., B. Ghorbani, S. Krishnan, et al. (2022). *Adaptive Gradient Methods at the Edge of Stability*. Preprint.

Cohen, J. M., S. Kaur, Y. Li, et al. (2021). "Gradient Descent on Neural Networks Typically Occurs at the Edge of Stability". In: *ICLR*.

Frankle, J., G. K. Dziugaite, D. M. Roy, and M. Carbin (2020). "Linear Mode Connectivity and the Lottery Ticket Hypothesis". In: *ICML*.

Jastrzebski, S., M. Szymczak, S. Fort, et al. (2020). "The Break-Even Point on Optimization Trajectories of Deep Neural Networks". In: *ICLR*.

Jordan, K., Y. Jin, V. Boza, et al. (2024). *Muon: An Optimizer for Hidden Layers in Neural Networks*. Blog post. URL: https://kellerjordan.github.io/posts/muon/.

Karpathy, A. (2022). *nanoGPT*. Software. URL: https://github.com/karpathy/nanoGPT.

Lanczos, C. (1950). "An Iteration Method for the Solution of the Eigenvalue Problem of Linear Differential and Integral Operators". In: *Journal of Research of the National Bureau of Standards* 45.4, pp. 255–282.

Soudry, D., E. Hoffer, M. S. Nacson, et al. (2018). "The Implicit Bias of Gradient Descent on Separable Data". In: *JMLR*.

Zhang, C., D. Zou, and Y. Cao (2024). "The Implicit Bias of Adam on Separable Data". In: *NeurIPS*.

A Hyperparameters

The values below are those of the configuration files and training code in the repository.

A.1 Runs of Figure 1

The 15 main TinyStories-10M runs (three optimizers, data orders d0–d4) share the following.

- **Model.** nanoGPT with pre-LayerNorm blocks: 6 layers, 6 heads, width 384, GELU MLPs of width 1536, no biases, no dropout, untied output head, separate query, key and value projections; context 256 tokens; 13.9M parameters (10.6M without the token and position embeddings and the head).
- **Data.** TinyStories (GPT-4 version) with a 4096-token byte-level BPE trained on the training split. A batch is 128 windows of 256 tokens at offsets drawn from the training split by a generator seeded with (data order, step), the same for every optimizer.
- **Length and schedule.** 10,000 steps, i.e. 328M tokens (about 0.6 epochs). Learning rate $\eta\,\varphi(t)$ at step t for every parameter group: linear warmup $\varphi(t) = (t+1)/200$ for $t < 200$, then cosine decay to $\varphi = 0.1$ at step 10,000. No gradient clipping.
- **Seeds and precision.** Initialization seed 0 for all runs; data orders 0–4. Training steps use bfloat16 autocast; predictions and Hessian measurements run without autocast, in float32 (TF32 matrix products allowed).
- **Logged predictions.** Next-token distributions on 128 fixed validation sequences × 16 evenly spaced positions (2,048 distributions, stored as float16 log-probabilities), and the loss on 64 fixed training sequences (all positions), used to match runs by loss; at every 250th step and at 30 log-spaced steps between 1 and 10,000.
- **Hessian.** At every 500th step and 30 log-spaced steps: top eigenvalue by 20 Lanczos iterations on 32 fixed training sequences, for AdamW also that of $P^{-1/2}HP^{-1/2}$.

Table 1: Optimizers of the TinyStories-10M runs. Learning rates: best final validation loss of a five-rate sweep on data order 0, except SGD, whose best rate (0.1) blew up on data orders 1–3, so it uses the next lower one.

Optimizer	η	Update	Weight decay	Parameters
SGD	0.03	heavy ball, $\mu = 0.9$, not Nesterov	none	all
AdamW	$2 \cdot 10^{-3}$	$\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$, bias-corrected moments	0.1, decoupled; none on LayerNorm gains	all
Muon	0.01	Nesterov momentum $\beta = 0.95$; 5 quintic Newton–Schulz steps in bfloat16; scale $\sqrt{\max(1, n_{\text{out}}/n_{\text{in}})}$	none	query, key, value, attention-output and both MLP matrices of every block
Muon's aux. AdamW	$2 \cdot 10^{-3}$	$\beta = (0.9, 0.95)$, $\epsilon = 10^{-8}$	none	embeddings, head, LayerNorm gains

A.2 Runs of Figure 2

The main runs of each setting (three optimizers, data orders d0–d4) are built as in Appendix A.1, except for the entries of Table 2 and the following.

- **Shakespeare-char.** Character-level, nanoGPT's 90/10 split; 24.6M tokens (about 24 passes over the 1M training characters). AdamW uses $\beta = (0.9, 0.99)$, as in nanoGPT; Muon's auxiliary AdamW keeps $(0.9, 0.95)$. Predictions on all 256 positions of 64 validation sequences (16,384 distributions) at every 50th step and 40 log-spaced steps; Hessian at every 50th step and 30 log-spaced steps.
- **TinyStories-30M.** 655M tokens (about 1.2 epochs). Predictions at every 500th and Hessian at every 1000th step, each with 30 log-spaced steps.
- **Shakespeare, long (memorization control).** AdamW and Muon only, data orders 0–4 (10 runs), as Shakespeare-char but 8,000 steps (131M tokens) with the cosine decay stretched to step 8,000; predictions at every 100th step and 40 log-spaced steps; no Hessian measurements.

Table 2: Settings of Figure 2. L layers, width d, vocabulary V; parameters in parentheses exclude the embeddings and the head; batches are sequences × tokens; w warmup steps; "aux.": learning rate of Muon's auxiliary AdamW. Learning rates are selected as in Table 1, for Shakespeare by the minimum validation loss over training (the runs overfit); SGD's best rate diverged on other data orders (0.2 on Shakespeare, 0.3 on TinyStories-30M), so it uses a lower one.

Setting	L, heads, d, V	Params	Steps	Batch	w	η_{SGD}	η_{AdamW}	η_{Muon} (aux.)
Shakespeare-char	6, 6, 384, 65	10.8M (10.6M)	1,500	64×256	100	0.05	$3 \cdot 10^{-4}$	0.04 (10^{-3})
TinyStories-10M	as in Appendix A.1							
TinyStories-30M	8, 8, 512, 4096	29.5M (25.2M)	20,000	128×256	400	0.1	$2 \cdot 10^{-3}$	0.01 ($1.5 \cdot 10^{-3}$)
Shakespeare, long	6, 6, 384, 65	10.8M (10.6M)	8,000	64×256	100	–	$3 \cdot 10^{-4}$	0.04 (10^{-3})

A.3 Runs of Figure 3

- **Parents.** The three d0 runs of Appendix A.1. During the run of optimizer A, the other two optimizers are stepped on the same mini-batch gradients with learning rate 0 (shadow states): their momenta, second moments and step counts evolve as if they were training, while the parameters follow A alone. At the fork steps 50, 100, 200, 400, 700, 1000, 1500, 2500 and 4000 the parent saves its parameters and the states of all three optimizers.
- **Swaps.** The fork $A \to B$ at step s loads the parameters at s and B's shadow state, uses B's hyperparameters (Table 1), and continues from s to step 10,000 on the same schedule (learning rate $\eta_B \, \varphi(s)$ at s) and the same batches (data order 0). Model, probes and logging grids are the parent's. All six ordered pairs at all nine steps: 54 forks.
- **Controls.** A continued with the same batches is the parent itself. A continued from step s with its own state but data order 1 is the data-order control: 27 forks, 81 in total.